NVIDIA X NEBIUS GLOBAL AI HACKATHON

AI Engineering Incident Investigator

An agentic AI platform that investigates software incidents by reasoning over source code, documentation, logs, GitHub issues, and historical incidents — and produces an evidence-backed root-cause analysis with a verified fix.

Primary track: Coding and Agentic Engineering | Secondary: Best Apps and Agents | Nebius + Nemotron powered

> **One-line description.** The system plans its own investigation, calls tools to gather evidence from code, docs, logs, commits and past incidents, forms and verifies a hypothesis, then generates a tested code patch — with citations and a confidence score at every step.

Stack at a glance: React + FastAPI + PostgreSQL / pgvector • RAG with reranking • Tool calling + MCP • Short-term and long-term memory • Evaluation and observability • Docker on Nebius AI Cloud.

Full project plan — revised edition. Two competition fixes are applied throughout and highlighted in yellow and green boxes: (1) all LLM calls run on Nebius with NVIDIA Nemotron open-source models, and (2) a new Fix Generator stage that writes, tests, and verifies a code patch.

Contents and executive summary

1. The problem	6. Agent system	11. Evidence and confidence	16. Database
2. What the user sees	7. Tool calling	12. Fix Generator (new)	17. Frontend
3. AI architecture	8. MCP	13. Evaluation system	18. Deployment on Nebius
4. Knowledge sources	9. Short-term memory	14. Observability	19. Final feature set
5. RAG system	10. Long-term memory	15. Authentication	20. Build progression
21. How this aligns with the NVIDIA x Nebius Global AI Hackathon + submission checklist			

Why this project wins attention

Most hackathon projects are chatbots that answer from retrieved text. This project is different in four ways. First, it solves a painful, real problem: production incidents cost engineering teams hours of manual searching across disconnected sources. Second, the AI does not just retrieve — it plans an investigation, chooses tools, compares evidence, and verifies its own hypothesis before answering. Third, every claim carries evidence and a confidence score, and facts are kept separate from inferences, so an engineer can trust and audit the result. Fourth, it closes the loop: it generates a code patch, runs tests in a sandbox, and drafts a pull request. That turns an analysis tool into a coding agent.

FIX 1 APPLIED — NEBIUS + NEMOTRON (MANDATORY)

All LLM calls run through **Nebius Token Factory** (or Nebius AI Cloud) using NVIDIA open-source **Nemotron** models. **Nemotron 3 Ultra** handles deep root-cause reasoning, hypothesis formation, and verification. **Nemotron Super** handles medium tasks such as evidence comparison and patch explanation. **Nemotron Nano** handles fast tasks such as incident classification, log triage, and summarization. This is stated in the architecture, deployment, and submission checklist so judges can verify it.

FIX 2 APPLIED — FIX GENERATOR (NEW STAGE)

A new agent stage produces a suggested code patch with a diff view, explains the change, runs relevant tests in a sandbox, verifies the fix against the evidence, and drafts a GitHub pull request description. New tools: `generate_patch()`, `run_tests()`, `verify_fix()`. This is what places the project firmly in the Coding and Agentic Engineering track.

1. The problem

Imagine a software company has a production issue: *"The payment API started returning 500 errors after yesterday's deployment. Find out why."*

Normally, an engineer must do all of this by hand:

1. Search the documentation.
2. Look through GitHub commits.
3. Inspect relevant source code.
4. Search previous incidents.
5. Look at logs.
6. Check configuration.
7. Compare recent changes.
8. Form a hypothesis.
9. Verify the hypothesis.
10. Write an incident report.

This takes hours, needs deep knowledge of the codebase, and is easy to get wrong under pressure. Key evidence is often missed because it lives in different tools. The AI Engineering Incident Investigator automates most of this investigation: it gathers the same evidence an engineer would, connects it, explains its reasoning, and proposes a verified fix.

Who it helps

- **On-call engineers** who need a fast, reliable first investigation.
- **New team members** who do not yet know the codebase history.
- **Engineering leads** who need a clear, evidence-backed incident report.

2. What the user sees

The user opens a web application, selects a project, and describes the incident in plain language. The investigation runs live on screen:

```
AI Incident Investigator
-----------------------------------------
Projects   |  Incident #1042
My Project |  "Payment API returning 500s"
Incidents  |
Documents  |  AI Investigation
GitHub     |  [done] Searched recent commits
Settings   |  [done] Retrieved payment documentation
           |  [done] Analyzed relevant code
           |  [done] Compared previous incidents
           |  [running] Investigating configuration changes
           |
           |  Root Cause: ...
           |  Evidence: ...
           |  Proposed Patch: ...  Test Result: PASSED
           |  Pull Request draft: ready
```

The final report shows the root cause, the evidence behind it (with links to the exact commit, document section, log line, or past incident), a confidence score, a proposed code patch in a diff view, the sandbox test result, and a ready-to-use pull request description.

The user can then ask follow-up questions:

- "Why do you think the database connection pool is responsible?"
- "Show me the evidence."
- "Was this problem seen before?"
- "Which commit introduced this?"
- "What should we change?"
- "Apply the patch and show me the test output."

3. AI architecture

The system is a layered agentic application. The user talks to a React frontend. A FastAPI backend runs an Agent Manager that coordinates retrieval, tools, and memory. All reasoning is done by Nemotron models served through Nebius.



How the layers work together

- **React UI** shows the live investigation, evidence, and report.
- **FastAPI** provides REST APIs, authentication, and streaming updates.
- **Agent Manager** plans the investigation, picks tools, tracks state, and decides when to stop.
- **RAG** retrieves relevant document and code chunks with citations.
- **Tools / MCP** fetch commits, files, issues, logs, and incident records, and later generate and test patches.
- **Memory** holds the current investigation (short-term) and durable project knowledge (long-term).

Figure 1 (next page): Full-page System Architecture Diagram shows all seven layers, the agent loop, the Nebius + Nemotron model layer, and the Sandbox. It is the visual summary of Sections 3–18.

FIX 1 APPLIED — MODEL LAYER

The generic "LLM" box is replaced by a **Nemotron model router** on **Nebius Token Factory**:

Model	Used for	Why
Nemotron 3 Ultra	Root-cause reasoning, hypothesis formation, hypothesis verification, final report	Deepest reasoning; the investigation's final judgment
Nemotron Super	Evidence comparison, patch explanation, PR description drafting	Strong mid-weight reasoning at lower cost
Nemotron Nano	Incident classification, log triage, query rewriting, summarization	Fast and cheap for high-volume small tasks

Every LLM call in the system goes through Nebius Token Factory or Nebius AI Cloud. No external proprietary LLM is used. Where helpful, Nebius Serverless Endpoints can host a dedicated model and Nebius Jobs can run batch evaluation.

System Architecture — AI Engineering Incident Investigator

Layered view of the full system. Straight vertical flow shows the investigation path; the side bar applies to every layer. Colours are consistent across this document.

USER — Engineer / On-call

↓

LAYER 1 • PRESENTATION — React

React UI — single-page app

Dashboard | **Investigation page — live agent steps** | Evidence viewer | Knowledge page | Evaluation page

↓ REST + WebSocket / SSE

LAYER 2 • API — FastAPI

FastAPI

Auth — JWT | Projects | Incidents | Investigations | **WebSocket / SSE — live progress**

↓

LAYER 3 • AGENT ORCHESTRATION — Agent Manager / LangGraph

Planner break incident into steps → **Investigator** call tools, gather evidence → **Hypothesis** most likely root cause → **Verifier** test hypothesis vs evidence → **Fix Generator** patch + test + verify → **Report Writer** evidence-backed report

Loop: Verifier ↻ Investigator if evidence is weak — agent re-investigates before reporting

↓ uses ↓

LAYER 4 • INTELLIGENCE — RAG + Tools + MCP + Memory

RAG
Parse → chunk → embed → pgvector search → rerank → citations

Tool Calling
`search_documents()`
`search_code()`
`get_file()`
`search_logs()`
`generate_patch()`

MCP Layer
GitHub MCP
Docs MCP
Incident MCP

Memory
Short-term: investigation state
Long-term: project facts, past incidents

↓ reasoning via ↓

LAYER 5 • MODEL LAYER — NEBIUS + NVIDIA NEMOTRON — Mandatory — Fix 1

Nebius Token Factory / Nebius AI Cloud — all LLM calls run here. No external proprietary LLM.

Nemotron 3 Ultra
Reasoning, root cause, verification

Nemotron Super
Evidence comparison, patch explanation

Nemotron Nano
Fast triage, summarisation, classification

Optional: Nebius Serverless Endpoints (dedicated hosting) • Nebius Jobs (batch evaluation)

↓ reads / writes ↓

LAYER 6 • DATA & EXTERNAL SYSTEMS — PostgreSQL + pgvector

PostgreSQL + pgvector
users, projects, incidents, chunks, memories, tool_calls

GitHub Repos
code, commits, issues, PR drafts

Documents & Logs
docs, simulated / structured logs, historical incidents

Sandbox — Fix Generator execution (Fix 2, green) — isolated container runs `generate_patch()` / `run_tests()` / `verify_fix()` . Generated code never touches production data. Result: patch diff + test output + PR draft back to Layer 3.

CROSS-CUTTING ALL LAYERS

Observability
Traces for every LLM, retrieval, tool, and MCP call. Latency, token use, cost per investigation.

Evaluation
`python evaluate.py`
Retrieval, answer, agent, patch correctness.

Auth & Security
JWT, per-user scoping, sandboxed test runs, secret handling.

Deployment
Docker + CI/CD
GitHub → Tests → Build → Deploy on Nebius AI Cloud.

Every box in Layers 1–6 emits traces and respects auth. Evaluation runs offline against the same stack.

Blue — core agent & UI **Yellow** — Nebius + Nemotron (Fix 1) **Green** — Fix Generator / Sandbox (Fix 2)
Dark — cross-cutting concerns

How to read this diagram — investigation flow 1 to 7. (1) Engineer describes the incident in the React UI. **(2)** FastAPI authenticates and streams live progress via WebSocket/SSE. **(3)** Agent Manager plans; Investigator gathers commits, code, docs, logs, and past incidents. **(4)** RAG, tools, MCP, and memory supply cited evidence. **(5)** Nemotron on Nebius reasons — Ultra verifies the hypothesis, Super explains the patch, Nano triages logs; weak evidence loops back to (3). **(6)** Fix Generator patches, Sandbox tests, verifier checks against evidence. **(7)** Report Writer returns root cause, confidence, patch diff, test results, and PR draft. Observability and evaluation cover every step.

4. Knowledge sources

The system ingests multiple types of information so the agent can reason like an engineer who knows the project.

Documentation

```
docs/
|-- architecture.md
|-- database.md
|-- authentication.md
|-- payments.md
|-- deployment.md
`-- api-reference.md
```

Source code

A GitHub repository is connected and indexed so the agent can search and read code:

```
src/
|-- auth/
|-- payments/
|-- users/
`-- database/
```

Issues, incident history, and logs

- **GitHub issues** are imported and searchable.
- **Incident history** is stored as structured records (INC-001, INC-002, ...) with symptoms, root cause, and fix.
- **Logs** start as simulated, structured logs for the demo; a real logging source can be integrated later without changing the agent.

5. RAG system

Retrieval-Augmented Generation (RAG) is one of the major components. It grounds every answer in real project material and provides citations.

Ingestion pipeline

```
Document -> Parsing -> Chunking -> Metadata extraction
         -> Embeddings -> Vector database (pgvector)
```

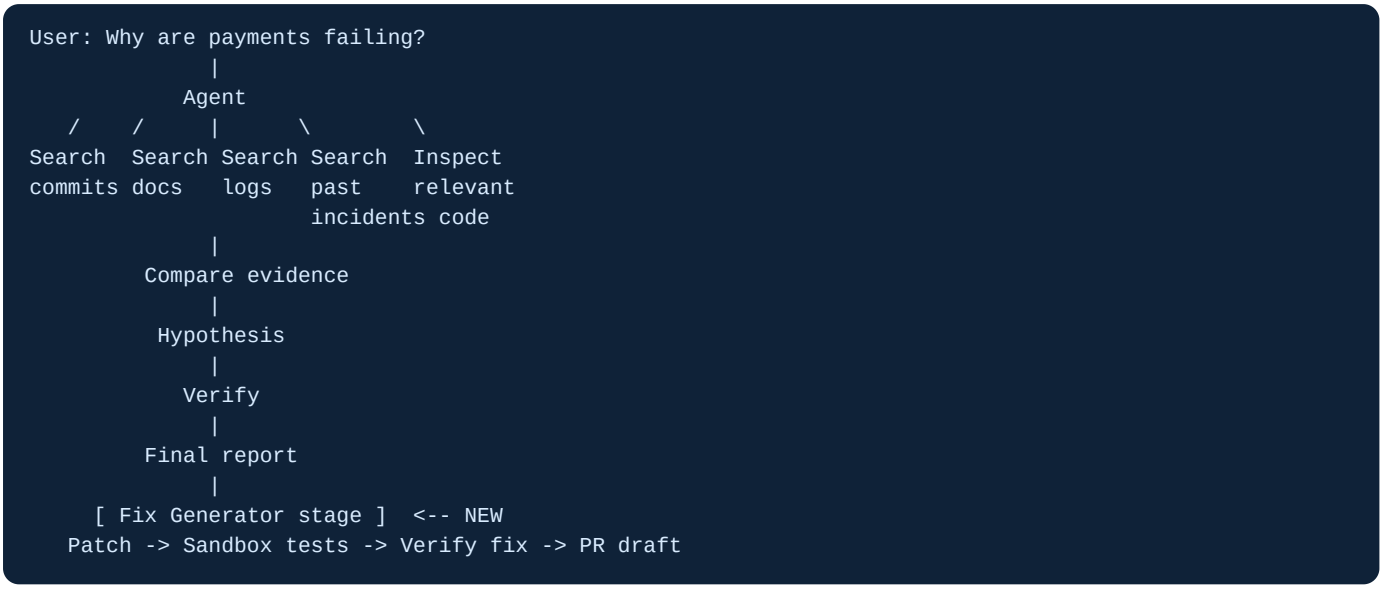
Query pipeline

```
Question -> Query embedding -> Vector search
         -> Relevant chunks -> Reranking / filtering -> LLM
```

Each retrieved chunk keeps its source metadata (file, section, commit, or incident ID). When the report makes a claim, the user can click **View evidence** and see exactly where the information came from. This makes the system far more credible than a normal chatbot, and it is what allows the evaluation system to check whether the right evidence was retrieved.

6. Agent system

The agent is the heart of the project. It does not simply retrieve documents and answer. It decides what to investigate, in what order, and when the evidence is strong enough.



Investigation loop

1. **Plan:** break the incident into investigation steps.
2. **Gather:** call tools to collect commits, docs, code, logs, and past incidents.
3. **Compare:** check evidence for consistency and contradictions.
4. **Hypothesize:** state the most likely root cause (Nemotron 3 Ultra).
5. **Verify:** actively look for evidence that would disprove the hypothesis.
6. **Report:** produce the evidence-backed report with confidence.
7. **Fix (new):** generate a patch, test it in a sandbox, verify it against the evidence, and draft a pull request.

> **FIX 2 APPLIED — AGENT WORKFLOW**
> The Fix Generator runs as a final agent stage, not a separate manual step. The agent only proposes a patch after the hypothesis is verified, and it only marks the fix as ready after sandbox tests pass. If tests fail, the agent revises the patch or clearly reports that no safe fix was found.

7. Tool calling

The LLM decides when to use each tool. Tools are small, typed functions with clear inputs and outputs.

Tool	Purpose
`search_documents()`	Search indexed documentation
`search_code()`	Search repository code
`search_github_commits()`	Find recent or relevant commits
`search_github_issues()`	Search GitHub issues
`get_file()`	Read a specific file
`search_logs()`	Search structured logs
`get_incident()`	Read one incident record
`search_previous_incidents()`	Find similar past incidents
`query_database()`	Run a safe, read-only query
`generate_patch()` **(new)**	Produce a code patch for the verified root cause
`run_tests()` **(new)**	Run relevant tests against the patch in a sandbox
`verify_fix()` **(new)**	Check the patch against the evidence and test output

8. MCP

Once tools work as normal functions, some are exposed through the Model Context Protocol (MCP), the standard protocol for connecting AI applications to external tools.



- **GitHub MCP** serves commits, files, issues, and pull request drafting.
- **Docs MCP** serves document and code search over the vector store.
- **Incident MCP** serves incident history and investigation records.

This demonstrates that the team understands how modern AI applications interact with external systems through a standardized protocol, and it makes the tools reusable by other MCP-compatible agents.

9. Short-term memory

The agent remembers the current investigation so follow-up questions work naturally.

```
User: Investigate the payment issue.
AI:   I found three suspicious changes.
User: Ignore the first one and investigate the second.
AI:   The second change modifies...
```

Short-term memory holds the conversation, the current hypothesis, the evidence collected so far, the patch state, and which steps are done. It lives for the duration of one investigation session.

10. Long-term memory

The system also remembers useful information across investigations, for example: "The payment service uses PostgreSQL," or "The team previously fixed a similar connection-pool issue in INC-017."

Not everything is stored. Long-term memory is curated into clear categories:

```
Memory
|-- user preferences
|-- project facts
|-- architectural decisions
|-- previous incidents
`-- investigation context
```

Each memory entry records where it came from, so the agent can cite it and a user can correct or delete it. This teaches when and why persistent memory is useful, and when it should not be trusted blindly.

11. Evidence and confidence

The AI never just says "the database is broken." Every investigation ends with a structured, auditable result:

```
Root Cause
--------------------------------------
Database connection pool exhaustion

Confidence
--------------------------------------
87%

Evidence
--------------------------------------
1. 23 connection timeout errors in logs
2. Connection pool changed in commit abc123
3. Previous incident INC-017 had the same symptoms
4. Configuration exceeds recommended limit
```

Facts versus inferences

Type	Example
Fact (directly observed)	"The pool size was changed from 20 to 100 in commit abc123."
Inference (reasoned conclusion)	"This change is likely contributing to the failures."

The confidence score is computed from the amount, quality, and agreement of the evidence — not invented by the model. Contradicting evidence lowers the score and is shown to the user. This separation of fact and inference is valuable AI engineering thinking and a clear differentiator for judges.

12. Fix Generator (new stage)

> **FIX 2 APPLIED — NEW SECTION**
> This stage turns the investigator into a coding agent. It is the main reason the project belongs in the **Coding and Agentic Engineering** track.

What it does

1. **Generate:** `generate_patch()` writes a minimal code patch for the verified root cause.
2. **Explain:** Nemotron Super explains the patch in plain language, file by file.
3. **Show:** the UI presents the patch as a diff view the engineer can read and copy.
4. **Test:** `run_tests()` runs the relevant tests against the patched code in an isolated sandbox.
5. **Verify:** `verify_fix()` checks the test output and confirms the patch addresses the evidence (for example, the pool limit is back within the recommended range).
6. **Draft:** the agent drafts a GitHub pull request description with the root cause, evidence, patch summary, and test results.

If tests fail, the agent says so plainly, shows the failure, and either revises the patch once or reports that no safe automated fix was found. The engineer always approves before anything is applied.

13. Evaluation system

An evaluation system separates this project from beginner AI projects. A test dataset is created with known answers:

```
Question:        What caused Incident #17?
Expected evidence: commit abc123, documentation section X
Expected conclusion: Connection pool exhaustion
Expected fix:      pool size restored to recommended limit
```

Running `python evaluate.py` automatically scores the system:

```
RAG Retrieval Accuracy:     91%
Evidence Grounding:         94%
Answer Accuracy:            88%
Tool Selection Accuracy:    92%
Patch Correctness:          85%   (new)
Sandbox Test Pass Rate:     90%   (new)
Average Latency:            4.2s
Average Cost:               $0.018
```

Area	What is measured
Retrieval	Did it retrieve the correct documents and relevant code?
Answer	Was the conclusion correct, supported by evidence, and free of hallucination?
Agent	Did it choose the right tools without unnecessary calls?
Fix (new)	Is the patch correct? Do sandbox tests pass? Does the fix match the evidence?
Performance	Latency, token usage, and cost per investigation

These numbers are generated from the test set, never faked. Failed cases are listed so the team can see exactly what to improve.

14. Observability

Every investigation is fully traced so the team can debug and improve the AI system itself:

```
Investigation ID
|-- LLM call (model, tokens, latency, cost)
|-- Retrieval (query, chunks returned, scores)
|-- Tool call (name, inputs, outputs, duration)
|-- MCP call (server, method, duration)
|-- Memory retrieval (entries used)
|-- Patch + test run (new)
`-- Final response
```

Tracked metrics include latency, errors, token usage, tool calls, retrieval results, model responses, and cost. This is production AI observability: if an answer is wrong, the trace shows whether the fault was retrieval, reasoning, tooling, or the patch stage.

15. Authentication

The application includes Register, Login, and Logout. Data is strictly scoped:

```
User -> Projects -> Repositories
                  -> Documents
                  -> Incidents
                  -> Investigations
```

A user can never access another user's projects, documents, or investigation history. API keys for GitHub and Nebius are stored securely per user and never exposed to the frontend. This gives the project real backend engineering depth beyond the AI layer.

16. Database — PostgreSQL + pgvector

PostgreSQL is the single database, with the pgvector extension for embeddings. This keeps the stack simple: one system stores application data, memory, and vectors.

Table	Holds
`users`	Accounts and credentials (hashed)
`projects`	User projects
`repositories`	Connected GitHub repositories
`documents` / `document_chunks`	Uploaded docs and their embedded chunks
`incidents`	Incident records and history
`investigations` / `messages`	Investigation sessions and conversation
`memories`	Curated long-term memory entries
`tool_calls`	Tool, MCP, patch, and test execution log
`evaluation_runs`	Evaluation scores over time

17. Frontend

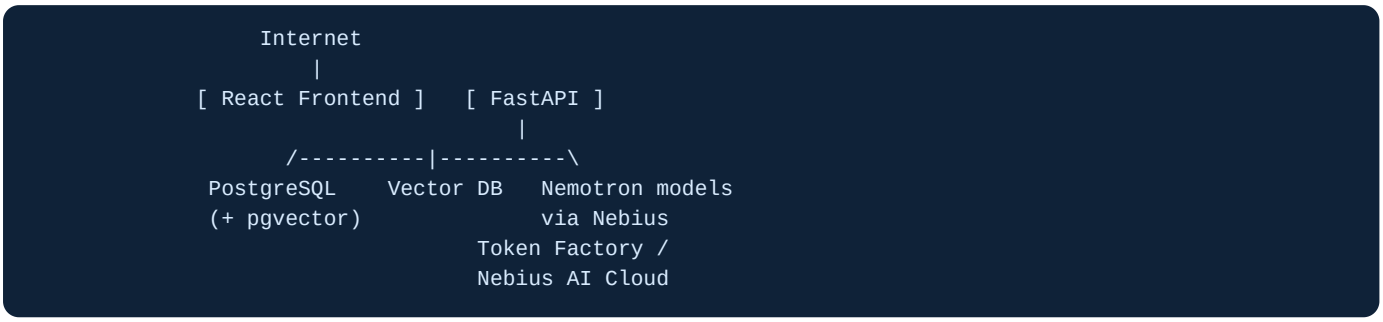
The frontend uses the team's existing React experience and gives it a clear purpose:

- **Dashboard:** projects, recent incidents, recent investigations, system statistics.
- **Investigation page:** question, live agent progress, evidence panel, reasoning summary, final report, proposed patch diff, and test results.
- **Knowledge page:** documents, repository, issues, and incidents.
- **Evaluation page:** retrieval score, answer score, patch correctness, test pass rate, latency, cost, and failure cases.

18. Deployment on Nebius

FIX 1 APPLIED — DEPLOYMENT

The whole system is deployed so that model inference runs on Nebius. This satisfies the mandatory competition rule and is stated in the README and demo.



- **Containerization:** the full stack is packaged with Docker.
- **CI/CD:** GitHub -> Tests -> Build -> Docker -> Deploy.
- **Model access:** Nemotron 3 Ultra, Super, and Nano are called through **Nebius Token Factory**. Optionally, a **Nebius Serverless Endpoint** hosts a dedicated model for predictable latency, and **Nebius Jobs** run batch evaluation over the test dataset.
- **Sandbox:** patch tests run in an isolated container so generated code can never touch production data.

19. Final feature set

Area	Features
AI	LLM integration (Nemotron on Nebius), prompt engineering, structured outputs, tool calling, agents, RAG, embeddings, vector search, reranking, short-term memory, long-term memory, MCP, multi-step workflows, patch generation, AI evaluation
Software engineering	Python, FastAPI, React, PostgreSQL, REST APIs, authentication, testing, Git/GitHub, Docker, CI/CD, cloud deployment
Production AI	Logging, tracing, token tracking, cost tracking, latency monitoring, evaluation, error handling, security, prompt/version management

This combination demonstrates software engineering plus AI engineering, rather than prompt engineering alone — a very strong Associate AI Engineer portfolio project.

20. Build progression

Every stage must be working software before moving to the next:

```
Basic Chat -> RAG -> Tool Calling -> Agent -> Memory
-> MCP -> Fix Generator -> Evaluation -> Production
```

1. **Basic Chat:** React + FastAPI + Nemotron on Nebius answering questions.
2. **RAG:** document ingestion, embeddings, cited answers.
3. **Tool Calling:** commits, code, issues, logs, incidents.
4. **Agent:** planned multi-step investigation with verification.
5. **Memory:** short-term session state, then curated long-term memory.
6. **MCP:** expose GitHub, Docs, and Incident tools via MCP.
7. **Fix Generator:** patch, sandbox tests, verification, PR draft.
8. **Evaluation:** test dataset and `evaluate.py` scoring.
9. **Production:** auth, observability, Docker, CI/CD, deployment.

21. How this aligns with the NVIDIA x Nebius Global AI Hackathon

Track fit

Track	Fit	Why
Coding and Agentic Engineering (primary)	Strong	The agent reasons over code, commits, and issues; calls tools; and now writes, tests, and verifies a code patch with a PR draft. That is a coding agent, not just a Q&A tool.
Best Apps and Agents (secondary)	Strong	A complete, useful agentic application powered by Nemotron models on Nebius, with a polished investigation UI.

MANDATORY RULE — SATISFIED

All submissions must run on Nebius Token Factory or Nebius AI Cloud and use at least one NVIDIA open-source model. This project uses **three** Nemotron models (Ultra, Super, Nano) through Nebius Token Factory, stated in the architecture (Section 3), deployment (Section 18), README, and demo video.

Judging criteria

Criterion	How this project scores
Technological Implementation	Three-tier Nemotron routing on Nebius, RAG with reranking and citations, tool calling, MCP servers, dual memory, sandbox patch testing, full evaluation and observability. Judges can verify Nebius/Nemotron usage in the repo and demo.
Design	Live investigation view with visible agent steps, an evidence panel with source links, fact-vs-inference separation, confidence display, and a readable patch diff with test results. The user always sees what the AI is doing and why.
Potential Impact	Production incidents are expensive and stressful. Cutting investigation time from hours to minutes helps every software team, especially small teams and new engineers without deep codebase history.
Quality of Idea	An investigator that verifies its own hypothesis, separates facts from inferences, scores its confidence, and then writes and tests the fix is a fresh, non-obvious idea — clearly beyond a standard RAG chatbot.

21. Alignment (continued) — submission checklist

Submission checklist

- [ ] Working demo URL (deployed app the judges can open)
- [ ] Demo video, 3 minutes or less, showing a full investigation ending with a patch and passing tests
- [ ] Public GitHub repository with an MIT or Apache-2.0 open-source license
- [ ] README that clearly highlights: Nemotron models used (Ultra / Super / Nano), Nebius Token Factory integration, architecture diagram, setup steps, and how to run evaluation
- [ ] Short feedback note on the Nebius and NVIDIA tools experience
- [ ] Evaluation results generated by `python evaluate.py` included in the README

> **Bottom line.** With the Nebius + Nemotron model layer and the Fix Generator stage applied, this idea aligns very strongly with the competition. It is a real agentic engineering product: it investigates, proves, fixes, and verifies — and every part of it can be shown live in under three minutes.